

TAD: Tokenizer Analysis Dashboard

CSC3003S Capstone Project
Week 1 Meeting

Introductions

- Supervisor/Client: Francois Meyer

Email: francois.meyer@uct.ac.za

Office: Room 314.10, Computer Science Building

- Tutor: Tadi Nyakonda

Email: NYKTAD002@myuct.ac.za

Today's session

- Background
 - Subword tokenization
 - Multilingual LLMs
 - Morphology
- System requirements



OpenAI
ChatGPT 4.0



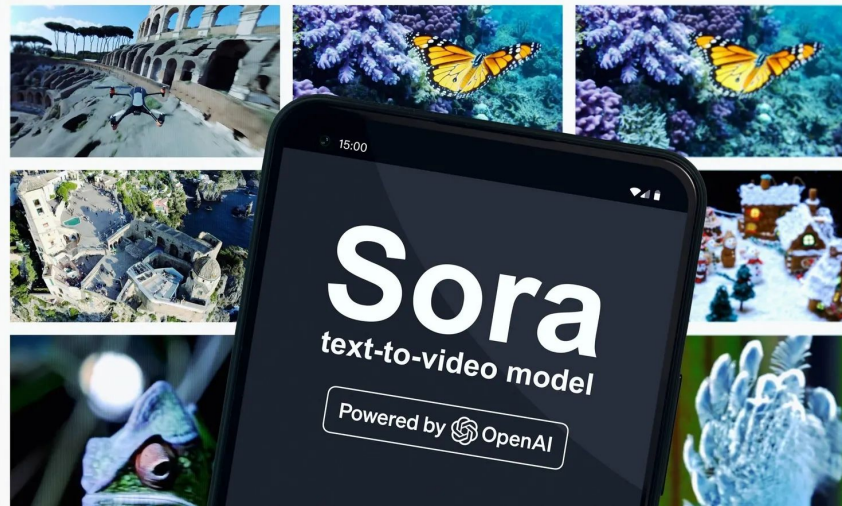
GitHub Copilot



Google
Translate



amazon alexa



Subword tokenization

Large language models (LLMs) like ChatGPT process text as a sequence of **subword tokens** .

- Before ChatGPT encodes your input prompt, the prompt is tokenized – some words are split into subwords.

Where is Cape Town? → Whe re _is _Cap e _Town ?

- When ChatGPT generates a response, it generates one subword token at a time.

Cape Town is in South Africa. → Cap e _Town _is _in _Sou th _Africa .

Subword tokenization

Why subwords instead of words/characters?

- Morphemes are the fundamental units of meaning in language.

unthinkable = un + think + able

untouchable = un + touch + able

- Enables the LLM to handle new words e.g. if you are told what the word “wug” means, you understand what “unwuggable” means.

Subword tokenization

Practical reasons

- Process previously unseen words (“out-of-vocabulary”) as sequence of known subwords.
- Construct meaning of rare words from meaning of their subwords.
- Handle misspellings (if one character is missing/wrong, the rest of the subwords are still the same).
- Leads to smaller vocabularies, so computationally efficient.

{cat, cats, dog, dogs, cow, cows} VS {cat, dog, cow, -s}

Subword tokenization

Modern subword tokenization is data-driven:

- Use data to tell use us how to tokenize.
- Ideally adapt to the properties of the language automatically.
- Tokens can be parts of words as well as whole words.

Subword tokenization

Modern subword tokenization is data-driven:

- Use data to tell use us how to tokenize.
- Ideally adapt to the properties of the language automatically.
- Tokens can be parts of words as well as whole words.

Different subword tokenization algorithms have been proposed:

- Byte-Pair Encoding ([BPE](#)) (Sennrich et al., 2016)
- Unigram language modeling tokenization ([ULM](#)) (Kudo, 2018)
- [WordPiece](#) (Schuster and Nakajima, 2012)

BPE: byte-pair encoding

Terminology

- **Training:** The data-driven process where the BPE algorithm builds its subword vocabulary and learns how to split words into subwords.
- **Corpus:** A collection of text data used in linguistics or NLP. It provides the basis for calculating subword frequencies in BPE.
- **Training corpus:** The specific text dataset used by BPE to learn and create the subword vocabulary.
- **Testing corpus:** A separate dataset used to evaluate the performance of the BPE tokenizer on new, unseen text.
- **Tokenization :** The process of transforming raw, unsegmented text into a sequence of subword tokens.

BPE

BPE and related algorithms have two parts:

- A **type learner (training)** that takes a raw training corpus and induces a vocabulary (a set of types).
- A **token segmenter (tokenization)** that takes a raw test sentence and tokenizes it according to that vocabulary.

BPE

Type learner

- Start with a vocabulary consisting of all individual characters
e.g. {A, B, C, D,..., a, b, c, d...}
- Repeat until k merges have been done:
 - Choose the two symbols that are most frequently adjacent in the training corpus (say “A”, “B”).
 - Add a new merged symbol “AB” to the vocabulary.
 - Replace every adjacent “A” “B” in the corpus with “AB”.

k is chosen to control the desired subword vocabulary size!

BPE

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ **to** k **do** # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

 Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

BPE example

STEP 1: Train BPE

Training corpus: the cat is in the hat

Vocabulary: [t, h, e, c, a, i, s, n]

BPE example

STEP 1: Train BPE

Training corpus: the cat is in the hat

Vocabulary: [t, h, e, c, a, i, s, n, at]

BPE example

STEP 1: Train BPE

Training corpus: the cat is in the hat

Vocabulary: [t, h, e, c, a, i, s, n, at, th]

BPE example

STEP 1: Train BPE

Training corpus: the cat is in the hat

Vocabulary: [t, h, e, c, a, i, s, n, at, th, the]

BPE example

STEP 1: Train BPE

Training corpus: the cat is in the hat

Vocabulary: [t, h, e, c, a, i, s, n, at, th, the]

STEP 2: Tokenize with BPE

Apply merges in same order as training until no more merges possible.

the c at i s i n the h at

BPE

- In practice, the most frequent words are included in the vocabulary, so not split into subwords, while rare words are split up.
- The subwords may or may not correspond to morphemes (lemmas or affixes).
- We don't have to deal with the out-of-vocabulary word problem anymore, as all words can be tokenized into subwords.

BPE

How to “untokenize” back into words?

the c at i s i n the h at

- To enable the algorithm to learn to represent the boundary between tokens, commonly a special end-of-word symbol (e.g. “_”, “@”, “Ġ”) is added before spaces in the training corpus.

the c at_ i s_ i n_ the_ h at_

- Or alternatively a start-of-word symbol is added between the space and the next word.

the Ġc at Ġi s Ġi n Ġthe Ġh at

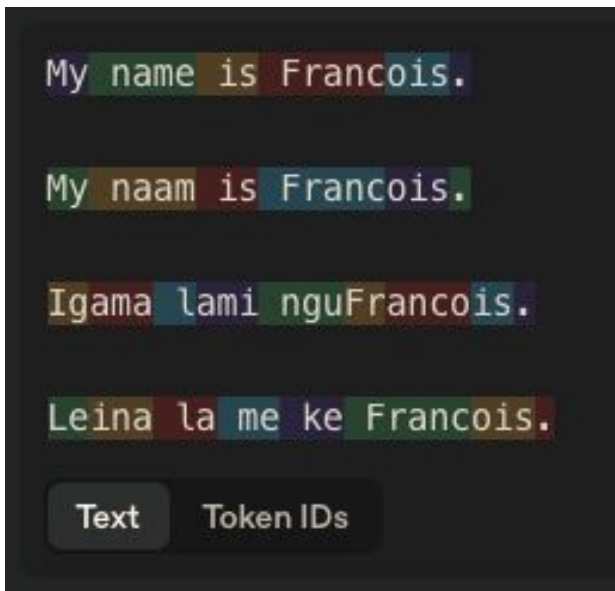
Why does tokenization matter?

Multilingual LLMs

Languages are not represented equally in LLM tokenizers.

Low-resource languages are split into more subword tokens.

- More tokens = **more expensive!**
- More tokens = **fewer words** in LLM vocabulary.



Do All Languages Cost the Same?

Tokenization in the Era of Commercial Language Models

Orevaoghene Ahia[◇] Sachin Kumar^{♦♥} Hila Gonen[◇] Jungo Kasai[◇]

David R. Mortensen[♦] Noah A. Smith^{◇♥} Yulia Tsvetkov[◇]

[◇]Paul G. Allen School of Computer Science & Engineering, University of Washington

[♦]Language Technologies Institute, Carnegie Mellon University

[♥]Allen Institute for Artificial Intelligence

{oahia,jkasai,nasmith,yuliats}@cs.washington.edu, sachink@allenai.org
hilagnn@gmail.com, dmortens@cs.cmu.edu

Abstract

Language models have graduated from being research prototypes to commercialized products offered as web APIs, and recent works have highlighted the multilingual capabilities of these products. The API vendors charge their users based on usage, more specifically on the number of “tokens” processed or generated by the underlying language models. What constitutes a token, however, is training data and model dependent with a large variance in the number of tokens required to convey the same information in different languages. In this work, we analyze the effect of this non-uniformity on the fairness of an API’s pricing policy across languages. We conduct a systematic analysis of the cost and utility of OpenAI’s language model API on multilingual benchmarks in 22 typologically diverse languages. We show evidence that speakers of a large number of the supported languages are overcharged while obtaining poorer results. These speak-

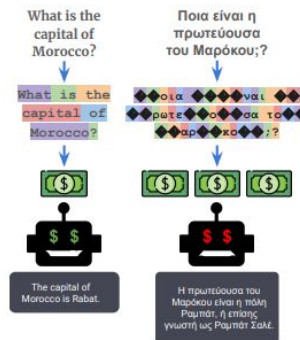


Figure 1: We investigate the effects of subword tokenization in LMs across languages with different writing systems. Our findings highlight disparities in the utility of LMs, as well as socio-economic disparities and increased costs in using commercial APIs for speakers of underrepresented languages.¹

from openly releasing them (Abdalla et al., 2023).

Subword vs morphological tokenisation

- Morphemes are the fundamental units of meaning in language.

unthinkable = un + think + able

untouchable = un + touch + able

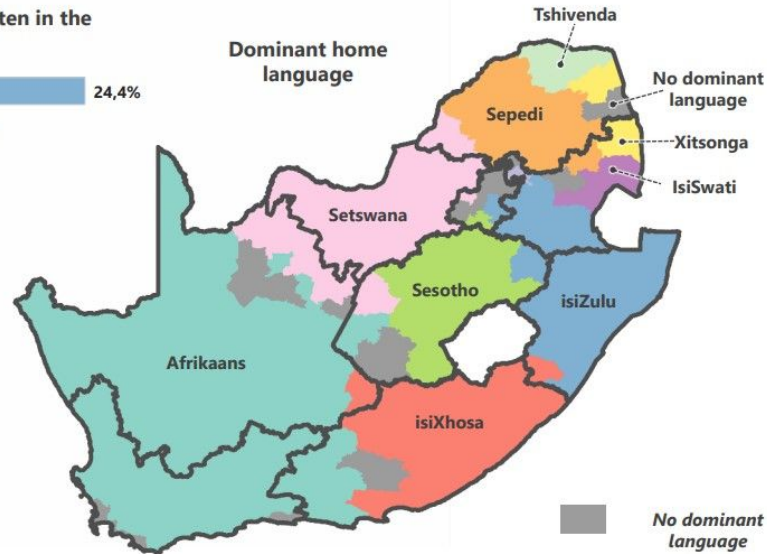
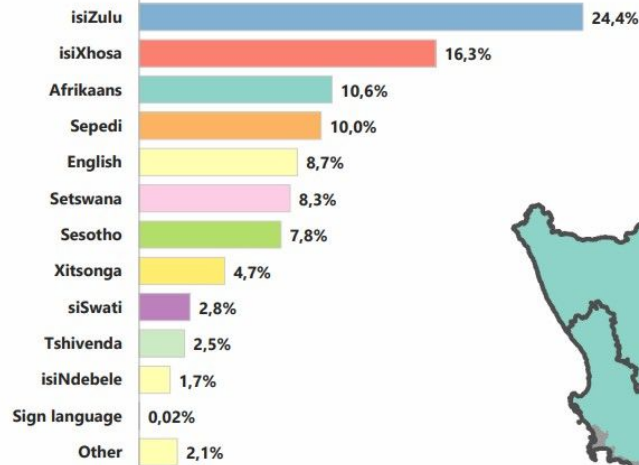
- Ideally, we want LLM tokenizers to come close to morphemes, but they don't – especially for low-resource languages e.g. isiNdebele:

BPE	Si -bu-ye si -tho-ko-ze khulu kwa-m-ani-k-elela em-phaka-thini ween-tjhaba-tjhaba nge -se-k-elo la-bo eli -ng-ana-ku-za-za em-za-bala-z-weni we-thu.
ULM	Si -bu-ye si -tho-ko-ze khulu kwama-nikele-la emphakathin-i w-eentjhabatjhab-a nge -se-ke-lo la-bo e-lingana-ku-za-za em-za-ba-la-zwe-ni we-thu.

What do I want?

A quick way to see how good a tokenizer is across all 11 official written South African languages.

Population distribution by language spoken most often in the household, 2022



What do I want?

Our languages are super diverse from linguistic perspective, so a great way to test the multilingual coverage of a tokenizer!

Low-resource

Nguni

isiZulu · isiXhosa · isiNdebele · siSwati

Conjunctive spelling, strongly agglutinative: long words, so subword tokenization is important.

Sotho-Tswana

Sepedi · Sesotho · Setswana

Disjunctive spelling: “words” look short, so not comparable to Nguni.

Other Niger-Congo B

Tshivenda · Xitsonga

Germanic

Afrikaans · English

Two tokenizers, one sentence — different splits

The same text tokenized by Gemma and Llama. Each coloured block is one token.

ENGLISH *the children are playing*

Gemma The _children _are _playing 4 tokens

Llama The _child ren _are _play ing 6 tokens

ISIZULU *izingane ziyadlala — ‘the children are playing’*

Gemma Iz ingane _ziya dlala 4 tokens

Llama I zing ane _ziy adl ala 6 tokens

SESOTHO *bana ba bapala — ‘children are playing’*

Gemma Bana _ba _bapala 3 tokens

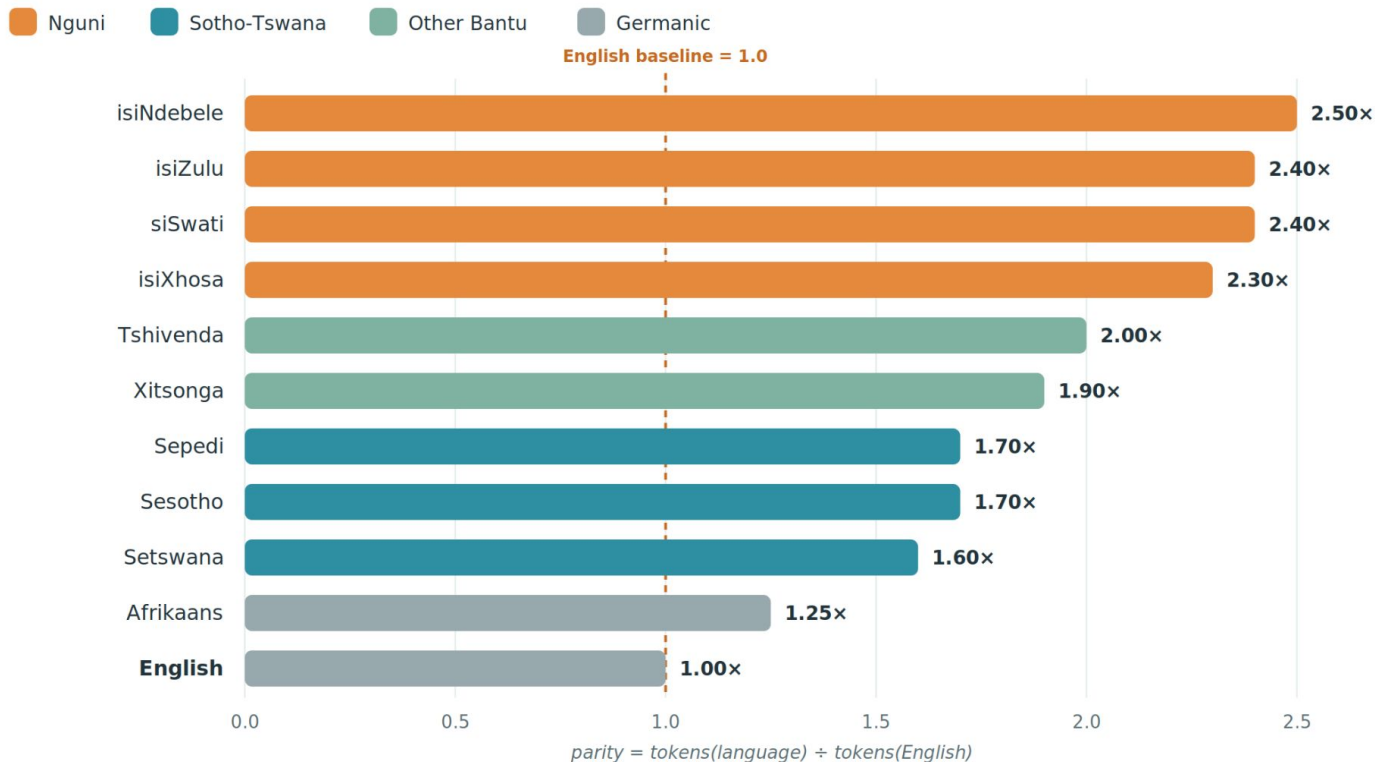
Llama Ban a _ba _bap ala 5 tokens

Illustrative splits. Same sentence → different tokenizers give different pieces and different counts; the gap widens for lower-resourced languages.

What do I want?

An interactive dashboard that I can use to analyse, compare, and rank different tokenizers based on efficiency and linguistic metrics across all 11 South African languages.

Metrics e.g. parity



Qualitative examples

Morphemes

(reference)

aba ntu ba ya hamba

Gemma

aba ntu ba ya hamba

5 / 5 aligned

Llama

Ab antu ba yah amba

1 / 5 aligned



token matches a morpheme (aligned)



boundaries don't match (misaligned)

System requirements

Requirements

1 Load any tokenizer: Let the user select tokenizers from Hugging Face models.

2 Apply the tokenizer to text in all 11 SA languages.

**TOKENIZER
ANALYSIS**

3 Present efficiency metrics e.g. fertility, token length, compression.

4 Present linguistic metrics e.g. morphological alignment.

5 Present qualitative examples e.g. where subwords = morphemes, where they don't.

Three dashboards

1. **INSPECT** a single tokenizer
2. **COMPARE** two tokenizers side by side
3. **RANK** several tokenizers on a leaderboard

1. INSPECT

Build an interactive dashboard that allows the user to:

- **Select one Huggingface tokenizer to inspect.**
- **Immediately present tokenizer analysis based on how the tokenizer tokenizes the default text corpora.**
- Allow the user to upload their own text corpora and re-run the tokenizer analysis based on that.
- Allow the user to type text into a text area and see/analyse how the tokenizer tokenizes it.

2. COMPARE

Build an interactive dashboard that allows the user to:

- **Select two Huggingface tokenizers to inspect.**
- **Immediately present a comparative tokenizer analysis based on how the tokenizers tokenize the default text corpora.**
- Allow the user to upload their own text corpora and re-run the comparative tokenizer analysis on that.
- Allow the user to type text into a text area and see/analyse how it's tokenized by the two tokenizers.

3. RANK

Build an interactive dashboard that allows the user to:

- **Continually add multiple tokenizers to a leaderboard that ranks tokenizers based on different metrics.**
- **Each tokenizer should be represented by a row of quantitative metrics that summarise its efficiency & linguistic metrics.**
- **This leaderboard should be permanent – if the user adds/removes tokenizers, changes must persist if the dashboard is restarted later.**

Tokenizer Analysis

Efficiency metrics:

- Token count
- Fertility (tokens per word)
- Compression (bytes per token)
- Token premium (tokens in language A / tokens in English on the same text)

Linguistic metrics:

- Morpheme boundary alignment (precision, recall, F1)

Features

- Analysis should provide a breakdown of each metric per-language.
- User should be able to filter between all 11 languages – select some, deselect others – and the analysis presentation should update accordingly.

Default datasets

Compute efficiency metrics on the FLORES dev dataset:

- Same 997 sentences translated into all 11 South African languages.

Compute linguistic metrics on the NCHLT dataset:

- A corpus in which linguists have manually split every word into morphemes.

GUI design

- Must be intuitive to understand the analysis.
- Must be concise – find a way to represent a LOT of statistics without forcing me to scroll for a long time.
- Examples:
 - Metrics / analysis / visualisations: <https://grubbr.app/>
 - Qualitative tokenization analysis: <https://platform.openai.com/tokenizer>
 - Leaderboard:
<https://huggingface.co/spaces/BabyLM-community/BabyLM-Leaderboard-2026>

Software engineering techniques

- This should be presented as a website that I can run locally on my laptop – and that you will run on your laptop for the demo.
- Implement the website however you want!
- Python for text processing and algorithms
- **Use the Huggingface transformers library for tokenizers.**

AI usage

- Make sure you understand the AI usage rules and requirements of CSC3003S.
- Include an AI declaration in your submissions: declare exactly how you use AI – which AI systems, for which components.
- **If you create a generic vibe-coded GUI, you will be get poor marks.**
- **Understand your code! If you can't explain your code, you will get very poor marks and could even fail.**

Project website

francois-meyer.github.io/tad

- Project information
- Resources
- Further reading
- FAQ

Questions?

Attendance

332	MSXKUN001	DLDTHA021	NKHLET004	
407	MHMUZA007	SWRPRA001	BMHCAV001	
415	NGWETH001	RNWMUK001	MDZMUT005	
434	DBNKYL001	SSMTIM001	PRTKEE002	
371	WBTLES001	KBLPAU002	RVHZWI001	
436	FKXMZW001	MBHKWA005	ABRSHI012	
387	PCKTIN001	ORLSTI001	WTRBEN002	